

WEEK – 3 (Skill)

Name: Anthony Nissy

Rollno: 2520030064

Section: 8

Experiment No. 5(Q5)

Title: Single and Double Quote Handling in Shell Parser using C

Aim: To implement single and double quote handling in a shell parser that preserves literal content, ignores variable expansion inside single quotes, stores quoted strings, preserves spaces inside double quotes, allows variable expansion, parses nested tokens, validates parsing results, and tests quoted commands and edge cases.

Objective:

Part 1 – Single Quotes

- To apply single quotes in shell commands.
- To preserve literal content inside single quotes.
- To ignore variable expansion inside single quotes.
- To store quoted strings correctly.
- To validate parsing results.
- To test different edge cases.

Part 2 – Double Quotes

- To apply double quotes in shell commands.
- To preserve spaces inside double quotes.
- To allow variable expansion.
- To parse nested tokens.
- To validate outputs.
- To test quoted commands.

Theory:

1. Single Quotes

Single quotes ' ' are used to preserve the literal content of a command.

Everything inside single quotes is treated as ordinary text.

For example:

```
echo 'Hello World'
```

is treated as one quoted string:

```
Hello World
```

The shell does not perform variable expansion inside single quotes.

2. Literal Content

Literal content means that the characters inside single quotes are preserved exactly as entered.

Example:

```
echo 'Hello $USER'
```

The \$USER is treated as normal text.

Output:

```
Hello $USER
```

Therefore, single quotes are useful when the exact content needs to be preserved.

3. Variable Expansion

Variable expansion means replacing a variable reference with its value.

For example:

```
NAME=Teja
```

```
echo $NAME
```

The output is:

```
Teja
```

But when the variable is inside single quotes:

```
echo '$NAME'
```

the output is:

```
$NAME
```

Therefore, variable expansion is ignored inside single quotes.

4. Storing Quoted Strings

A shell parser should store the complete content inside quotes as one token.

Example:

```
echo 'Operating Systems'
```

The tokens are:

```
echo
```

```
Operating Systems
```

The space between Operating and Systems does not create another token.

5. Parsing Validation

The parser should verify whether the single quote has a corresponding closing quote.

Valid command:

```
echo 'Hello World'
```

Invalid command:

```
echo 'Hello World
```

Output:

Error: Missing closing single quote.

6. Edge Cases

The parser should also test different edge cases such as:

```
echo "
```

```
echo '$USER'
```

```
echo 'Hello World'
```

```
echo 'Hello $USER'
```

```
echo 'Hello World
```

These cases help verify correct quote handling and syntax validation.

7. Double Quotes

Double quotes " " are used to preserve spaces inside a string while allowing variable expansion.

Example:

```
echo "Hello World"
```

The complete string Hello World is treated as one token.

8. Preserving Spaces

Without quotes:

```
echo Hello World
```

the parser can identify:

```
echo
```

Hello

World

But:

```
echo "Hello World"
```

should be treated as:

```
echo
```

```
Hello World
```

Thus, double quotes preserve spaces inside the quoted string.

9. Variable Expansion in Double Quotes

Unlike single quotes, double quotes allow variable expansion.

Example:

```
NAME=Teja
```

```
echo "Hello $NAME"
```

Output:

```
Hello Teja
```

The \$NAME is replaced by its value.

10. Nested Tokens

Double quotes can contain multiple words and variables.

Example:

```
echo "Hello $USER Welcome"
```

The complete quoted portion is treated as one token.

The parser first identifies:

```
Hello $USER Welcome
```

and then performs variable expansion.

11. Validation of Double Quotes

The parser checks whether the opening double quote has a closing double quote.

Valid:

```
echo "Hello World"
```

Invalid:

```
echo "Hello World
```

Output:

Error: Missing closing double quote.

Algorithm:

Single Quote Handling

- Start the program.
- Read a command from the user.
- Scan the input character by character.
- Check for a single quote '.
- When a single quote is found, start collecting characters.

- Continue until the closing single quote is found.
- Store the quoted content as one token.
- Preserve the literal content.
- Ignore variable expansion.
- Check for a missing closing quote.
- Display the generated token.
- Validate the parsing result.
- Stop.

Double Quote Handling

- Start the program.
- Read a command from the user.
- Scan the input character by character.
- Check for a double quote ".
- Start collecting characters inside the quotes.
- Preserve spaces inside the quoted string.
- Check for variable references beginning with \$.
- Expand the variable using its stored value.
- Continue until the closing double quote is found.
- Store the complete quoted string as one token.
- Validate the output.
- Detect missing closing quotes.
- Display the parsing result.
- Stop.

Functions Used:

Function	Purpose
main()	Starting point of the program
printf()	Displays messages and output
fgets()	Reads the command entered by the user
strcspn()	Removes the newline character
strlen()	Finds the length of a string

isspace()	Checks whitespace characters
getenv()	Retrieves the value of an environment variable
strcmp()	Compares strings
strcpy()	Copies strings
strcat()	Combines strings
return	Terminates the function/program

Sample Output:

Single Quote Test

Enter command: echo 'Hello World'

--- Token Stream ---

Token 1: echo

Token 2: Hello World

--- Parsing Result ---

Single quote detected.

Literal content preserved.

Variable expansion ignored.

Variable Expansion Test

Enter command: echo '\$USER'

--- Token Stream ---

Token 1: echo

Token 2: \$USER

--- Parsing Result ---

Single quote detected.

Variable expansion ignored.

Double Quote Test

Enter command: echo "Hello World"

--- Token Stream ---

Token 1: echo

Token 2: Hello World

--- Parsing Result ---

Double quote detected.

Spaces preserved.

Double Quote with Variable

Enter command: echo "Hello \$USER"

--- Token Stream ---

Token 1: echo

Token 2: Hello Teja

--- Parsing Result ---

Double quote detected.

Spaces preserved.

Variable expansion performed.

Invalid Single Quote

Enter command: echo 'Hello World

--- Syntax Validation ---

Error: Missing closing single quote.

Invalid Double Quote

Enter command: echo "Hello World

--- Syntax Validation ---

Error: Missing closing double quote.

Observation:

The program successfully identifies single and double quoted strings. Single quotes preserve the literal content and ignore variable expansion. Double quotes preserve spaces and allow variable expansion. The parser also validates quoted strings and detects missing closing quotation marks

Result:

The single and double quote handling functionality was successfully implemented. The program correctly preserved literal content, ignored variable expansion inside single quotes, stored quoted strings, preserved spaces inside double quotes, allowed variable expansion, parsed quoted tokens, validated outputs, and detected invalid quoted commands.

Part 2 Design and Implementation of Double Quote Handling, Variable Expansion and Quoted Token Parsing

Aim:. To design parser logic for double-quoted commands, preserve spaces, allow variable expansion, parse nested tokens, validate outputs, and test quoted commands.

Commands / Functions Used

1. gcc

gcc is used to compile the C source code into an executable program.

gcc quotes.c -o quotes

It helps verify that the implementation does not contain compilation errors.

2. ./quotes

This command executes the compiled shell parser program.

./quotes

It allows the user to enter commands and observe quote handling and parsing results.

3. printf()

printf() displays:

- Input prompts
- Token information
- Parsing results
- Error messages
- Validation messages

4. fgets()

fgets() reads the complete command entered by the user, including spaces.

This is important because quoted commands can contain multiple words.

5. `getenv()`

`getenv()` retrieves the value of an environment variable.

Example:

```
getenv("USER");
```

If the variable contains Teja, the value returned is:

Teja

6. `isspace()`

`isspace()` checks whether a character is whitespace such as a space or tab.

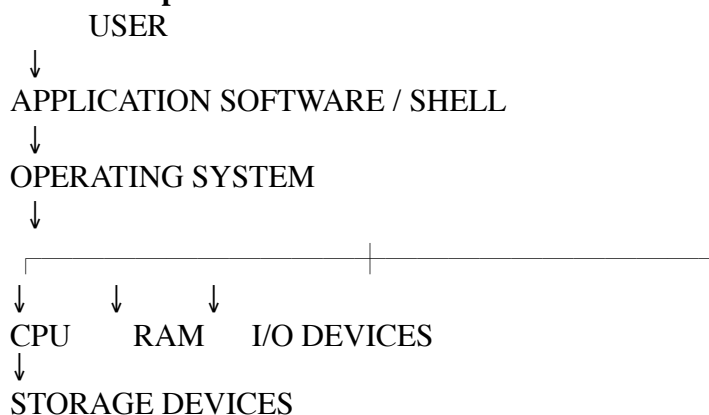
It helps the parser distinguish normal spaces from spaces that occur inside quotes.

Hardware and Operating System Relationship

Hardware refers to the physical components of a computer, such as the CPU, RAM, hard disk/SSD, keyboard, mouse, and display.

The Operating System acts as an interface between the user/application software and hardware. It manages hardware resources and provides services to programs. This follows the same hardware–OS relationship section used in the template.

Relationship



How OS interacts with hardware

- **CPU Management** – OS schedules processes and allocates CPU time.
- **Memory Management** – OS allocates and manages RAM.
- **File Management** – OS manages files stored on HDD/SSD.
- **Device Management** – OS controls devices such as keyboard, mouse, printer and display.
- **Process Management** – OS creates, executes and terminates processes.
- **Security** – OS controls access to hardware and system resources.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
```

```
#define MAX_INPUT 200
#define MAX_TOKENS 50
```

```

#define MAX_TOKEN 100

void printToken(int number, char token[])
{
    printf("Token %d: %s\n", number, token);
}

int main()
{
    char input[MAX_INPUT];
    char tokens[MAX_TOKENS][MAX_TOKEN];

    int count = 0;
    int i = 0;

    printf("Enter command: ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    while (input[i] != '\0')
    {
        /* Ignore whitespace outside quotes */
        if (isspace((unsigned char)input[i]))
        {
            i++;
            continue;
        }

        /* Single Quotes */
        if (input[i] == '"')
        {
            int j = 0;

            i++;

            while (input[i] != '\0' && input[i] != '"')
            {
                if (j < MAX_TOKEN - 1)
                    tokens[count][j++] = input[i];

                i++;
            }

            tokens[count][j] = '\0';

            if (input[i] == '"')
            {
                i++;
                count++;
            }
        }
    }
}

```



```

    }
    else
    {
        printf("Error: Missing closing single quote.\n");
        return 0;
    }
}

/* Double Quotes */
else if (input[i] == '"')
{
    int j = 0;

    i++;

    while (input[i] != '\0' && input[i] != '"')
    {
        if (input[i] == '$')
        {
            char variable[50];
            int k = 0;

            char *value;

            i++;

            while (input[i] != '\0' &&
                    (isalnum((unsigned char)input[i]) ||
                     input[i] == '_'))
            {
                if (k < 49)
                    variable[k++] = input[i];

                i++;
            }

            variable[k] = '\0';

            value = getenv(variable);

            if (value != NULL)
            {
                int v = 0;

                while (value[v] != '\0' &&
                        j < MAX_TOKEN - 1)
                {
                    tokens[count][j++] = value[v++];
                }
            }
        }
    }
}

```

```

    }
    else
    {
        if (j < MAX_TOKEN - 1)
            tokens[count][j++] = input[i];

        i++;
    }
}

tokens[count][j] = '\0';

if (input[i] == '"')
{
    i++;
    count++;
}
else
{
    printf("Error: Missing closing double quote.\n");
    return 0;
}
}

/* Normal Token */
else
{
    int j = 0;

    while (input[i] != '\0' &&
           !isspace((unsigned char)input[i]) &&
           input[i] != '\"' &&
           input[i] != '"')
    {
        if (j < MAX_TOKEN - 1)
            tokens[count][j++] = input[i];

        i++;
    }

    tokens[count][j] = '\0';
    count++;
}
}

printf("\n--- Token Stream ---\n");

for (i = 0; i < count; i++)
{
    printToken(i + 1, tokens[i]);
}

```

```
}

printf("\n--- Parsing Result ---\n");
printf("Quoted strings processed successfully.\n");

return 0;
}
```

OUTPUT: